However, if an error is thrown, such as a `ThreadDeath` or an `OutOfMemoryError`, the `ActorSystem` is terminated ungracefully by the JVM. Alternatively, you can set `akka.jvm-exit-on-fatal-error = off` in the configuration to try to exit the `ActorSystem` gracefully before terminating the JVM. Chapter 7 covers how configuration works.

To handle specific exceptions, you need to add them as a type of `.onFailure`, as shown in the following snippet. In this example, you catch only `IndexOutOfBoundException` and its subtypes using a resume strategy:

```
val behavior: Behavior[Command] = ...

Behaviors.supervise(behavior)
     .onFailure[IndexOutOfBoundException](SupervisorStrategy.resume)
```

Exceptions not of this type or its descendants are not caught by this strategy and result in the termination of the actor.

5.3.2 Stopping and the PostStop signal

By default, this is the actor supervisor strategy: if you don’t explicitly set a supervision strategy, one will be set for you. Stopping is the safest choice because you never know why the error occurred. Was there something in the message combined with the actor’s application logic? Could it have been a call to an external service? In previous versions of Akka, restart was a default strategy, but that was too optimistic. After a failure, it is safer to assume that the exception will not go away but will persist.